

Installation Manual

Purpose

This document describes the installation and configuration process for group one's 4P02 Group Project system. It reflects the codebase as of April 23, 2026. The database must be hosted using [Supabase](#) and while not mandatory, the recommended cloud host for the front end, python bridge, and backend is [Railway](#).

The system is made up of four major parts:

- Front End: React + Vite web client
- Database: Supabase SQL schema, RLS policies, database function, and edge functions
- Python Bridge: Python listener that subscribes to Supabase realtime events
- Engine: Java tokenization and comparison engine exposed to Python through Py4J

How The Software Is Stitched Together

1. Front end
 - Students submit .zip files through the frontend. The frontend:
 - i. strips comments and merges supported source files by language,
 - ii. uploads the processed archive to the Submissions storage bucket,
 - iii. encrypts student name and number through the encryptDecrypt edge function,
 - iv. generates a stable student identity hash through createStudentIdentityKey,
 - v. inserts a File_Submissions_New row through the create_file_submission database function.
 - Instructors can upload:
 - i. boilerplate files
 - ii. repository archives
 - Instructors can export a repository
 - Instructors can trigger a comparison which runs the createComparison edge function.
 - Once a comparison is marked as completed the frontend reads the stored comparison JSON files and displays results to the instructor.
2. Repository
 - The frontend authenticates users against Supabase Auth and reads/writes project data through Supabase.
 - Repository uploads invoke the importRepository edge function, which expands a master repository zip into imported submission rows in File_Submissions_New.

- The createComparison edge function gathers the latest submission per student, optionally includes imported repository submissions, and inserts a Comparisons row with status pending.
- 3. Python Bridge and Backend
 - Listens for inserts on:
 - i. Comparisons
 - ii. File_Submissions_New
 - For each new submission, the listener downloads the zip from Submissions, sends it to the Java engine through Py4J, receives a condensed token CSV, and uploads that CSV into the Tokens bucket.
 - For each comparison, the listener waits until all required token CSV files exist, optionally tokenizes the selected boilerplate file, calls the Java comparison engine, uploads per-submission comparison output to the Comparisons bucket, and updates the comparison row to completed.

Site Preparation

The following services should be prepared before the front end or backend is deployed

1. Supabase Project

Create one Supabase project for the application. The project must have:

- Auth enabled
- Realtime enabled for the public schema tables used by the listener
- Storage buckets created
- SQL schema applied
- RLS policies applied
- Edge functions deployed

2. Required Storage Buckets

Create these bucket names exactly as written:

- Submissions
- Tokens
- Comparisons
- Boiler_Plate
- Repositories

3. Network / Host Preparation

For a deployed installation:

- The Python listener and Java bridge must be able to run continuously on either a local machine or cloud host. Our recommended host is [Railway](#).
- The listener must have outbound access to the Supabase project URL

- The frontend must have access to the same Supabase project URL and anon/publishable key

4. Runtime Prerequisites

If running locally install the following:

- Node.js 20+ and npm
- Python 3.12
- Java JDK 17+ recommended

If you are using Railway the above do not need to be manually installed and come preloaded. However, the below are recommended regardless of if you are running it locally or on the cloud:

- Supabase CLI for deploying functions and running database migrations manually

Database Installation

[Supabase](#) must be used as the database host for this project.

The Database folder in the project repository contains:

- table definitions
- row-level security policies
- database functions
- edge functions

Create your supabase project first then apply the database objects in this order:

1. Create the tables
2. Create the database function
3. Apply the RLS policies
4. Deploy the edge functions. Make sure the following exist in the edge function secrets:
 - SUPABASE_URL
 - SUPABASE_ANON_KEY
 - SUPABASE_SERVICE_ROLE_KEY
 - STUDENT_IDENTITY_SECRET

Frontend Installation

The following are steps to set up the front end on Railway. Be advised that if a different cloud host is used the steps may differ slightly.

1. Ensure that you have the project repository on github, and that the repository you are the owner of the repository you are planning on using.
2. Create a new project on Railway.
3. Add a service referencing the github repository for the project.
4. Go to the settings for that service and set the root directory to “/FrontEnd”.

5. Then set the following environment variables:
 - VITE_SUPABASE_URL=your_supabase_project_url
 - VITE_SUPABASE_PUBLISHABLE_DEFAULT_KEY=your_supabase_publishable_key
6. Deploy the site.

The front end is now live, the domain for the site can be edited under settings.

Backend and Python Bridge Installation

The backend and python bridge are deployed together and started using docker. The docker file:

- starts from python:3.12-slim
- installs a headless default JDK
- installs PythonBridge requirements
- compiles Java sources
- starts Bridge and PythonBridge/listener.py

All libraries used by Java are pre installed inside the repository.

The following are steps to set up the backend and listener on Railway. Be advised that if a different cloud host is used the steps may differ slightly.

1. Inside the same project as the front end add another service linking to the project's github repository.
2. In the settings for the new service scroll down to Build, and change the builder from Railpack to docker, leaving the docker file path as the default.
3. Add the following environment variables:
 - URL=your_supabase_project_url
 - SECRET_KEY=your_supabase_service_role_key
4. Deploy the service.

The backend and python bridge are now live, you can view the logs by inspecting the backend deployment.

First-Time Verification

After installation, verify the system in this order:

1. Log into the frontend or create a new account.
2. Create a course and an assignment.

3. Submit a test student zip file.
4. Confirm a row is inserted into the File_Submissions_New table.
5. Confirm the Python listener uploads a token CSV to
Tokens/<assignment_id>/<submission_id>.csv.
6. Upload an optional boilerplate file or repository archive if needed.
7. Create a comparison from the instructor UI.
8. Confirm a row is inserted into Comparisons with status pending, then updated to
completed.
9. Confirm result files appear in the Comparisons bucket as
<comparison_id>/<submission_id>.json.
10. Open the comparison UI and verify the result renders.